



YallaMarket

يلا ماركت

Software Requirements Specification (SRS)

Version	V1.0
Date	March 2026
Course	Software Engineering
Methodology	Agile / Scrum
Status	Draft

Contents

1. Introduction	4
1.1 Preface	4
1.2 Project Overview	4
1.3 Scope	4
1.4 Definitions and Acronyms	4
1.5 References	5
1.6 Requirements document variability	5
2. System Overview	6
2.1 System Description	6
2.2 System Architecture	6
2.3 User Classes	6
2.4 Operating Environment	6
3. Functional Requirements & Requirements Prioritization	7
3.1 User Authentication	7
3.2 Product Catalog	7
3.3 Shopping Cart	7
3.4 Order Management	8
3.5 Admin Dashboard	8
4. Non-Functional Requirements	9
5. Use Case Summary	10
6. System Constraints	11
6.1 Technical Constraints	11
6.2 Project Constraints	11
6.3 Security Constraints	11
7. Data Requirements	12
7.1 Data Entities	12
User	12
Product	12
Order	12
8. Assumptions and Dependencies	14
8.1 Assumptions	14
8.2 Dependencies	14
9. Agile Development Plan (Scrum)	15
9.1 Team Roles	15
9.2 Sprint Plan	15

10. Appendix.....	16
10.1 API Endpoint Summary	16
10.2 Document Revision History.....	16

1. Introduction

1.1 Preface

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for YallaMarket (يلا ماركت), an Arabic-language online grocery ordering web application. This document serves as the primary reference for the development team, stakeholders, and academic evaluators throughout the project lifecycle.

1.2 Project Overview

YallaMarket is a web-based digital marketplace that allows users to browse and purchase digital products such as cards, subscriptions, and electronic balances.

The platform is inspired by Biddimarket.ps and provides a modern Arabic and English user interface. Users can explore categories, view product details, add items to a cart, and proceed to checkout.

1.3 Scope

The system covers the following areas:

- Online storefront with Arabic and English interface (RTL/LTR)
- Product catalog organized by categories
- Product details and related products
- Shopping cart and checkout flow
- Local state management for cart (frontend)
- Future support for authentication and backend integration

1.4 Glossary

Term / Acronym	Definition
SRS	Software Requirements Specification
RTL	Right-to-Left — text direction used in Arabic
MERN	MongoDB, Express.js, React.js, Node.js — the tech stack
API	Application Programming Interface
JWT	JSON Web Token — used for user authentication
CRUD	Create, Read, Update, Delete — basic database operations
ILS / ₪	Israeli Shekel — currency used for product pricing

Admin	A privileged user with access to the management dashboard
FR	Functional Requirement
NFR	Non-Functional Requirement

1.5 References

- Biddimarket.ps — Inspiration and UI reference
- IEEE Std 830-1998 — IEEE Recommended Practice for SRS
- React & SQLite Documentation
- Tailwind CSS v3 Documentation
- React Router DOM v6 Documentation

1.6 Requirements document variability

This document follows a structured requirements specification approach suitable for a medium-scale system. The level of detail provided reflects a plan driven development style. Systems developed incrementally may require less initial detail, with requirements evolving over time. This document is informed by IEEE standards for requirements documentation, which provide guidelines for large-scale software engineering projects.

2. System Overview

2.1 System Description

YallaMarket is a web-based grocery ordering platform designed for Arabic-speaking users. The system consists of two main parts: a React-based frontend that provides the user interface, and a SQLite backend that handles business logic and data persistence.

2.2 System Architecture

The system follows a three-tier architecture:

Layer	Technology	Responsibility
Presentation	React 18 + Vite + Tailwind CSS	User interface, RTL layout, routing
Application	Node.js + Express.js (not sure yet)	Business logic, REST API, authentication
Data	SQLite	Data storage, persistence, queries

2.3 User Classes

The system serves three distinct user types:

User Type	Description	Access Level
Guest	Unauthenticated visitor browsing the store	Browse products and categories only
Registered User	Authenticated customer with an account	Full shopping: cart, checkout, order history
Admin	Store manager with elevated privileges	Full access including dashboard and management

2.4 Operating Environment

- Web browsers: Chrome, Firefox, Edge, Safari (latest versions)
- Mobile-responsive design for screens from 375px to 1440px+
- Frontend hosted on Vercel (or similar)
- Backend hosted on **Render** (or similar)
- Database: MongoDB Atlas
- Internet connection required

3. Functional Requirements & Requirements Prioritization

3.1 User Authentication

ID	Feature	Description	Priority
FR-01	Register	Users shall create an account with name, email, phone, and password	High
FR-02	Login	Registered users shall log in using email and password	High
FR-03	Logout	Authenticated users shall log out and clear their session	High
FR-04	Auth Guard	Protected routes shall redirect unauthenticated users to the login page	High

3.2 Product Catalog

ID	Feature	Description	Priority
FR-05	Browse Categories	Users can view all product categories on the home page as tab cards with icon and name	High
FR-06	View Products	Users can view all products in a selected category in a responsive grid	High
FR-07	Search Products	Users can filter products by name in real time within a category page	Medium
FR-08	Product Detail	Users can view full product details including image, name, price, unit, and description	High
FR-09	Product Availability	Products marked as unavailable are hidden from the catalog	Medium

3.3 Shopping Cart

ID	Feature	Description	Priority
FR-10	Add to Cart	Users can add products to cart from product cards or the product detail page	High
FR-11	View Cart	Users can view all cart items on the /cart page with image, name, price, and quantity	High
FR-12	Update Quantity	Users can increase or decrease item quantity using +/- controls (minimum 1)	High

FR-13	Remove Item	Users can remove individual items from the cart	High
FR-14	Cart Summary	Cart page shows subtotal, delivery fee (₪5), and total price	High
FR-15	Persist Cart	Cart contents are saved in localStorage and restored on page reload	Medium
FR-16	Cart Badge	Navbar shows a badge with the total number of cart items	Medium

3.4 Order Management

ID	Feature	Description	Priority
FR-17	Place Order	Authenticated users can place an order by providing delivery address; payment is cash on delivery	High
FR-18	Order History	Users can view their past orders including date, items, and total	High

3.5 Admin Dashboard

ID	Feature	Description	Priority
FR-20	Manage Products	Admin can add, edit, and delete products including image upload	High
FR-21	Manage Categories	Admin can add and edit product categories	Medium
FR-22	Manage Orders	Admin can view all orders	High
FR-23	Dashboard Stats	Admin dashboard shows total orders, revenue, products count, and users count	Low
FR-24	Manage Users	Admin can view the list of registered users	Low

4. Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system — how it performs, not what it does. They are measurable system properties, not source code.

ID	Category	Requirement	Metric / Target
NFR-01	Performance	Pages must load within 3 seconds on a standard broadband connection	< 3 seconds
NFR-02	Performance	API endpoints must respond within 500ms under normal load	< 500ms
NFR-03	Usability	All UI text must be in Arabic with correct RTL layout	100% RTL
NFR-04	Usability	The interface must be fully responsive on mobile (375px) and desktop (1440px)	All breakpoints
NFR-05	Usability	Users must be able to complete a purchase in fewer than 5 steps	≤ 5 steps
NFR-06	Security	User passwords must be hashed using bcrypt before storage	bcryptjs
NFR-07	Security	Authentication tokens (JWT) must expire after 7 days	7-day expiry
NFR-08	Security	All API inputs must be validated server-side to prevent injection attacks	express-validator
NFR-09	Security	Admin routes must be protected and inaccessible to regular users	Middleware guard
NFR-10	Reliability	The system must handle API errors gracefully without crashing the UI	try/catch + UI feedback
NFR-11	Reliability	Cart data must persist across browser sessions using localStorage	localStorage
NFR-12	Maintainability	Code must be organized in clearly named folders following the defined project structure	Defined structure
NFR-13	Maintainability	Environment variables (DB URI, JWT secret) must never be hardcoded	.env file
NFR-14	Scalability	The backend must support adding new product categories without code changes	Data-driven
NFR-15	Compatibility	The system must work on Chrome, Firefox, Edge, and Safari latest versions	Cross-browser

5. Use Case Summary

UC ID	Use Case	Actor	Pre-condition	Post-condition
UC-01	Register Account	Guest	User not logged in	Account created, user logged in
UC-02	Login	Registered User	Valid credentials exist	Session started, JWT issued
UC-03	Browse Categories	Guest / User	None	Category page displayed
UC-04	Search Products	Guest / User	User on category page	Filtered product list shown
UC-05	Add to Cart	Guest / User	Product is available	Item added, cart updated
UC-06	View Cart	Guest / User	Cart has items	Cart page displayed
UC-07	Place Order	Registered User	Logged in, cart not empty	Order saved, cart cleared
UC-08	View Order History	Registered User	User is logged in	Order list displayed
UC-09	Manage Products	Admin	Admin logged in	Product added/edited/deleted
UC-10	Update Order Status	Admin	Admin logged in	Order status updated

6. System Constraints

6.1 Technical Constraints

- The frontend must be built using React 18 and Tailwind CSS v3 — no other UI frameworks
- The backend must use Node.js with Express.js — no other server framework
- The database must be MongoDB via Mongoose — no relational databases
- All prices must be displayed in Israeli Shekel (₪)
- The entire UI must be in Arabic with RTL layout

6.2 Project Constraints

- Three-week development
- Team of three members — each must have individual Git commits
- Must follow Scrum methodology with JIRA task tracking
- Must include proper software engineering documentation (UML, ERD, SRS)

6.3 Security Constraints

- No sensitive data (passwords, tokens) stored in plain text
- MongoDB URI and JWT secret must be stored in environment variables only
- Admin panel must not be accessible to non-admin users

7. Data Requirements

7.1 Data Entities

The system stores and manages the following main entities in **MongoDB**:

User

Field	Type	Constraints	Description
name	String	Required	Full name of the user
email	String	Required, Unique	Login email address
phone	String	Optional	Contact phone number
password	String	Required, Hashed	bcrypt hashed password
role	Enum	user admin	Access level
createdAt	Date	Auto	Account creation date

Product

Field	Type	Constraints	Description
nameAr	String	Required	Arabic product name
nameEn	String	Optional	English product name
price	Number	Required, > 0	Price in ILS (₪)
unit	String	Required	e.g. "500 غرام 1", "كيلو"
image	String	Optional	Image filename (Multer upload)
category	ObjectId	Ref: Category	Foreign key to Category
stock	Number	Default: 100	Available quantity
isAvailable	Boolean	Default: true	Visibility in catalog

Order

Field	Type	Constraints	Description
user	ObjectId	Ref: User	Customer who placed the order
items	Array	Required	[[product, quantity, priceAtOrder]]
totalPrice	Number	Required	Total in ILS including delivery

deliveryAddress	Object	Required	{fullName, phone, city, street}
status	Enum	Default: pending	pending processing shipped delivered cancelled
paymentMethod	String	Default: cash_on_delivery	Payment method used
createdAt	Date	Auto	Order placement timestamp

8. Assumptions and Dependencies

8.1 Assumptions

- Users have a stable internet connection during the session
- All users can read Arabic — no multi-language support is required in v1
- Payment is cash on delivery only — no payment gateway is integrated in v1
- Product images will be provided by the admin after launch
- The platform targets users in Palestine using ILS currency

8.2 Dependencies

Dependency	Version	Purpose
React	18.x	Frontend UI framework
Vite	5.x	Frontend build tool
Tailwind CSS	3.x	Utility-first CSS styling
React Router DOM	6.x	Client-side routing
Node.js	20.x	Backend runtime
Express.js	4.x	Backend web framework
MongoDB Atlas	Cloud	Cloud database service
Mongoose	8.x	MongoDB object modeling
JWT	9.x	User authentication tokens
bcryptjs	2.x	Password hashing
Lucide React	Latest	Icon library
react-hot-toast	2.x	Toast notification system
Cairo (Google Fonts)	Latest	Arabic-friendly web font

9. Agile Development Plan (Scrum)

9.1 Team Roles

Role	Responsibility	Assigned To
Scrum Master / Lead Dev	Setup, architecture, integration, JIRA management	Baraa Masri
Frontend Developer	UI components, pages, cart, routing	All the team using github
Backend Developer	API, database models, authentication	All the team using github

9.2 Sprint Plan

Sprint	Duration	Goals	Deliverable
Sprint 1	Week 1	Project setup, SRS, ERD, frontend layout, Navbar, Home page, Category page	Working UI skeleton
Sprint 2	Week 2	Product cards, Cart, Auth pages, Backend models, Auth API, Products API	Core features done
Sprint 3	Week 3	Orders API, Admin dashboard, Frontend-backend integration, Testing, Deployment	Live application

10. Appendix

10.1 API Endpoint Summary

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	None
POST	/api/auth/login	Login and receive JWT	None
POST	/api/auth/logout	Logout and clear token	User
GET	/api/auth/me	Get current user info	User
GET	/api/products	Get all products (filter by category, search)	None
GET	/api/products/:id	Get single product	None
POST	/api/products	Create product	Admin
PUT	/api/products/:id	Update product	Admin
DELETE	/api/products/:id	Delete product	Admin
GET	/api/categories	Get all categories	None
POST	/api/categories	Create category	Admin
POST	/api/orders	Place new order	User
GET	/api/orders/my	Get current user orders	User
GET	/api/orders	Get all orders	Admin
PUT	/api/orders/:id/status	Update order status	Admin

10.2 Document Revision History

Version	Date	Changes	Author
1.0	March 2026	Initial SRS document created	Team